合肥师范学院·考点对应模式 · 答案与解析

一、单项选择题（15×2=30分）

1.

答案：B

解析： - B 正确：冯·诺依曼结构以存储器为中心，指令和数据存放在同一个存储器中，按地址访问，这是"存储程序"的核心体现。 - A 错误：冯·诺依曼结构以存储器（而非运算器）为中心，且指令和数据共用同一存储器（不分离存储是哈佛结构的特点）。 - C 错误：存储程序思想是指程序和数据预先存入存储器后自动执行，而非运行时逐条手动输入。 - D 错误：冯·诺依曼结构的基本组成确实包含运算器、控制器、存储器和 I/O 设备，但"程序不能被修改"是错误的——存储程序的关键优势之一恰恰是程序可以像数据一样被读取和修改。

2.

答案：B

解析： - B 正确：编译程序（Compiler）将高级语言源程序整体翻译成目标程序，翻译完成后目标程序可直接运行，不依赖源程序。 - A 错误：汇编程序（Assembler）是将汇编语言源程序翻译成机器语言，而非高级语言源程序。 - C 错误：解释程序（Interpreter）是逐条解释执行源程序，不生成独立的目标程序，运行速度较慢。 - D 错误：反汇编程序是将机器语言程序转换为汇编语言程序，方向相反，不用于高级语言翻译。

3.

第 1 步：取绝对值并转二进制

$|-75| = 75$

将 75 转换为 7 位二进制：

$75 = 64 + 8 + 2 + 1 = 2^6 + 2^3 + 2^1 + 2^0$

$75_{(10)} = 1001011_{(2)}$

第 2 步：确定符号位

−75 为负数 → 符号位 = **1**

第 3 步：组合原码

原码 = 符号位 + 数值位 = **1** + **1001011** = **11001011**

答案 A = 11001011

A. 11001011

4.

答案：B

解析： - B 正确：n 位补码（含 1 位符号位）的表示范围为 -2^(n-1) ～ +2^(n-1)-1，当 n=8 时为 -128 ～ +127。 - A 错误：-127 ～ +127 是原码和反码的表示范围，补码比它们多表示一个最小负数 -128。 - C 错误：+128 超出了 8 位补码的正数范围（最大为 +127）。 - D 错误：-127 是原码/反码的下界，+128 超出上界，两个边界都错。

5.

第一步：将操作数转换为变形补码（符号位扩展）

对 4 位补码做符号位扩展，正数的符号位 0 扩展为 00：

操作数	4 位补码	变形补码（符号位扩展）
A = +5	0101	**00**101
B = +4	0100	**00**100

第二步：变形补码加法

```
  00101    ([A]补 = +5)
+ 00100    ([B]补 = +4)
-------
  01001    (5 + 4 = 9, mod 32 = 9)
```

结果码：**01001**

第三步：提取双符号位并判定溢出

结果码 01001 的最高 2 位为双符号位 $\mathbf{S_1S_0 = 01}$。

- $S_1 = 0$，$S_0 = 1$
- $S_1 \neq S_0$ → **双符号位相异，发生了溢出**
- 双符号位为 01 → **正溢出**（两个正数相加，结果超出正上限 +7）

判定规则对照：

双符号位	含义
00	结果为正，未溢出
01	**正溢出 ← 本题结果**
10	负溢出
11	结果为负，未溢出

场景规则验证

1. **溢出必然发生**：A=+5 与 B=+4 同号（均为正），A+B=9 > +7（超出 4 位补码表示范围上界），满足溢出条件。
2. **双符号位状态合法**：结果双符号位 01 ∈ {01, 10}（相异=溢出），合法。
3. **结果唯一确定**：双符号位 01 → 正溢出，唯一。

双符号位为 01，发生了正溢出。

正确选项：A

6.

A. 65436

1. 参数（PARAMS）

参数	值
short 位数 WIDTH	16
模 MOD = 2^16	65536
short 变量初值 S	−100
abs(S)	100

2. 核心建模（PROCESS）

步骤 1 — 求 S 的 16 位补码位模式

`short` 采用补码表示，S = −100 的 16 位补码位模式为：

$bits = S \bmod 2^{16} = (-100) \bmod 65536 = 65436$

二进制表示：`1111 1111 1001 1100`

步骤 2 — 同宽度转换 = 位模式重解读

`short` → `unsigned short` 是 16 位 → 16 位转换，**位模式完全不变**，仅按无符号语义重新解读。因此：

$u = bits = 65436$

步骤 3 — 验证公式

负数 S（S < 0）的 16 位补码位模式按无符号解读的数值满足：

$U = 2^{16} + S = 65536 + (-100) = 65436$

与步骤 2 的结果一致。

3. 结果（RESULT）

$\boxed{u = 65436}$

`printf("%u\n", u)` 输出 **65436**。

4. 场景规则验证（RULE_CHECK）

规则	验证	结果
规则1：位模式不变	bits(S) = 65436, u = 65436	相同 √
规则2：U == MOD + S	u = 65436, MOD + S = 65436	相等 √

5. 选项分析（OPTIONS）

选项	实际标的值	正误	说明
A	**65436**	✅ 正确	位模式重解读，U = 65536 + (−100) = 65436
B	−100	❌	忽略 unsigned 语义，直接保留原负值，未做位模式重解读
C	100	❌	误取绝对值，误以为转换仅去掉符号位
D	65336	❌	错误补码公式 65536 − 2×100 = 65336，多减了一次

四选项互异：A=65436、B=−100、C=100、D=65336，四个值各不相同，无碰撞。

7.

答案：C

解析： - C 正确：存储层次结构的核心目标是利用不同层次存储器的性能差异，在成本合理的条件下使整体速度接近 Cache、容量接近辅存，性价比最优。 - A 错误：扩大容量是辅存的功能之一，但不是层次化结构的主要目的，主要目的是速度-容量-成本的整体平衡。 - B 错误：降低功耗和简化设计并非层次结构的目标。 - D 错误：不同层次的访问方式并不统一（如主存按地址访问、磁盘按块/扇区访问），CPU 的访存逻辑也并非因此简化。

8.

1. **寻址单元数** = 2^n = 2^16 = **65536**（16 根地址线可寻址 65536 个存储单元）。
2. **总位数** = 寻址单元数 × 数据线位数 = 65536 × 8 = **524288 bit**。
3. **按字节编址**：每地址 1 字节（1 B = 8 bit），所以总容量（字节）= 总位数 ÷ 8 = 524288 ÷ 8 = **65536 B**。
4. **换算**：65536 B ÷ 1024 = **64 KB**。

A. 64 KB

9.

答案：B

解析： - B 正确：DRAM 刷新按行进行，由刷新控制电路自动完成，刷新期间 CPU 不参与控制，只需在刷新周期内让出总线。 - A 错误：DRAM 的刷新不是由 CPU 控制的，而是由专门的刷新逻辑电路定时控制；虽然读出时有"再生"效应，但刷新是独立的定时操作。 - C 错误：DRAM 的电容会不断泄漏电荷，必须在刷新周期内（通常 2ms）反复刷新才能保持数据，不能只刷新一次。 - D 错误：DRAM 刷新是按行进行的，与行地址直接相关，而非按列逐位。

10.

A. 0x803C

---

1. 已知参数

参数	值	说明
R_1	0x8000	变址寄存器
PC	0x2004	程序计数器（当前指令地址）
A	0x003C	形式地址字段
地址空间上限	0xFFFF	64KB，16位地址

2. 变址寻址定义

变址寻址的有效地址计算公式：

$EA = R_1 + A$

3. 计算有效地址

$EA = R_1 + A = 0x8000 + 0x003C = \textbf{0x803C}$

对应选项 **A**。

选项	值	错误类型	计算路径
A ✓	0x803C	—（正确）	变址寻址 EA = R_1 + A = 0x8000 + 0x003C = **0x803C**
B	0x003C	直接寻址混淆	误取 EA = A = 0x003C，忽略了变址寄存器的作用
C	0x2040	相对寻址混淆	误取 EA = PC + A = 0x2004 + 0x003C = 0x2040，错用了 PC 作为基址
D	0x7FC4	运算方向错误	误取 EA = R_1 − A = 0x8000 − 0x003C = 0x7FC4，加法误做减法

各干扰项为什么错：

- **B（0x003C）**：直接寻址的定义是 EA = A，但题目明确为**变址寻址**，有效地址必须加上变址寄存器 R_1 的值。
- **C（0x2040）**：相对寻址 EA = PC + A，但题目是变址寻址，基址是变址寄存器 R_1 而非程序计数器 PC。
- **D（0x7FC4）**：变址寻址的运算方向是**加法**（$R_1 + A$），而非减法（$R_1 - A$）。

11.

答案：B

解析： - B 正确：指令寄存器（IR）用于存放当前从主存取出的指令，供指令译码器译码和执行部件使用。 - A 错误：程序计数器（PC）存放的是下一条要执行的指令的地址，而非指令本身。 - C 错误：存储器地址寄存器（MAR）存放的是 CPU 要访问的主存单元地址，不是指令。 - D 错误：程序状态字寄存器（PSW）存放的是 CPU 的工作状态和标志位（如进位、溢出等），不是指令。

12.

答案：C

解析： - C 正确：指令周期定义为 CPU 从主存取出一条指令并执行完毕所需的全部时间。 - A 错误：时钟周期是计算机最基本的时钟信号周期，是最小的时间单位，通常小于机器周期。 - B 错误：机器周期通常指完成一个基本操作（如访存）所需的时间，一个指令周期通常包含若干个机器周期。 - D 错误：微周期（微指令周期）是执行一条微指令所需的时间，是最小执行单位之一，远小于指令周期。

13.

答案：B

解析： - B 正确：链式查询方式只用很少几根控制线（总线请求、总线忙、总线许可），线路简单、易于扩充，但离控制器物理位置越远的设备优先级越低（"排队"效应）。 - A 错误：链式查询中设备优先级与物理位置有关，不是公平的（公平的是计数器定时查询或独立请求方式）。 - C 错误：各设备拥有独立的请求线和许可线是独立请求方式的特点，而非链式查询。 - D 错误：任何仲裁方式都需要控制信号线，"无需任何控制信号线"的描述不成立。

14.

答案：C

解析： - C 正确：DMA 方式通过 DMA 控制器直接控制主存与高速外设之间的数据交换，以数据块为传输单位，CPU 只在传输开始（预处理）和结束（后处理）时介入，传输过程中 CPU 不干预。 - A 错误：DMA 方式适用于高速外设（如磁盘）的大批量数据传输，低速字符设备通常用中断方式。 - B 错误：DMA 方式在传输开始和结束时仍需要 CPU 参与（初始化参数、处理中断等），不

能说"完全不需要任何干预"。 - D 错误：DMA 方式的数据传输速率高于中断方式，因为数据传输不需要 CPU 逐字节参与。

15.

- **B（最常见错误路径）**：混淆了"中断隐指令保存断点"与"中断服务程序保存通用寄存器/现场"。约 30%–40% 考生会误选 B，因为两者都是"保存"，但保存的对象和执行主体完全不同。
- **C**：对中断机制完全错误的认知——用户程序在 CPU 响应中断时已被暂停，不可能自行感知中断并保存断点。
- **D**：将中断保存断点与函数调用（CALL/RET）机制完全等同——函数调用是软件（指令）行为，发生在执行阶段；中断保存断点是硬件自动行为，发生在响应阶段。

A. 由中断隐指令（硬件自动）在中断响应阶段完成保存断点

> 中断隐指令三项操作：关中断、保存断点（保存 PC/返回地址）、引出中断服务程序。保存断点归属：**中断隐指令（硬件自动，中断响应阶段）**。

二、填空题（10分）

1.

答案：运算器、控制器

解析： CPU 的核心组成是运算器（ALU 及相关寄存器）和控制器（产生控制信号的部件）。运算器负责算术运算和逻辑运算，控制器负责协调和控制各部件按指令要求工作。

2.

答案：触发器、电容

解析： SRAM（静态随机存储器）利用双稳态触发器存储信息，只要不断电就能稳定保持数据，不需要刷新，但集成度较低、功耗较大。DRAM（动态随机存储器）利用 MOS 管栅极电容上的电荷来存储信息，由于电容会自然泄漏电荷，必须定期刷新（通常间隔 2ms）才能维持数据正确。

3.

答案：1、8、23

解析： IEEE 754 单精度浮点数共 32 位，最高位为符号位（S），占 1 位；中间为阶码（E），采用移码表示，占 8 位；最低位为尾数（M），采用隐含最高位 1 的规格化表示，占 23 位。

4.

参数（PARAMS）

参数	含义	取值
L	指令总长度	16 位

参数	含义	取值
addr_len	地址码位数	4 位
N2	二地址指令条数	3
N1	一地址指令条数	8

逐级扩展（PROCESS）

二地址级：

· 二地址指令有 2 个地址码，操作码位数 = L − 2×addr_len = 16 − 8 = **8 位**

· 二地址操作码总编码数 = 2^8 = **256**

· 占用 N2 = 3 条后，剩余编码数 R2 = 256 − 3 = **253**

一地址级：

· 一地址可分配空间 = R2 × 2^addr_len = 253 × 16 = **4048**

· 占用 N1 = 8 条后，剩余编码数 R1 = 4048 − 8 = **4040**

零地址级：

· 零地址最大条数 = R1 × 2^addr_len = 4040 × 16 = **64640**

答案（RESULT）

\boxed{64640}

零地址指令最多有 **64640** 条。

5.

答案：组合逻辑（硬布线）、微程序

解析：　控制器按实现方式分为两种：组合逻辑控制器（也称硬布线控制器），用逻辑门电路直接组合产生控制信号，速度快但设计复杂、不易修改；微程序控制器，将控制信号编成微指令存放在控制存储器中，通过执行微程序来产生控制信号，设计规整、易于修改和扩充，但速度相对较慢。

三、简答题（5×6=30分）

1.

PARAMS（已知参数）

指令类型	CPI	使用占比
A 类指令	1	35%
B 类指令	3	30%

指令类型	CPI	使用占比
C 类指令	6	25%
D 类指令	10	10%

- 程序指令总数 N = 100000
- 主频 f = 2.5 GHz

占比之和 = 35% + 30% + 25% + 10% = 100%（归一满足）

第1问（3分）：计算该程序运行时的平均 CPI

公式： CPI_avg = Σ(CPI_i × 占比_i)，其中占比取小数形式。

逐项加权：

指令类型	计算	结果
A 类	1 × 0.35	0.35
B 类	3 × 0.30	0.90
C 类	6 × 0.25	1.50
D 类	10 × 0.10	1.00

求和：

$CPI_{avg} = 0.35 + 0.90 + 1.50 + 1.00 = 3.75$

> **第1问答案：平均 CPI = 3.75**

第2问（3分）：计算程序在该 CPU 上的执行时间（μs，保留两位小数）

公式： T = (N × CPI_avg) / f

单位换算： 主频 f = 2.5 GHz = 2.5×10^9 Hz = 2.5×10^9 s^{-1}

代入计算：

T = \frac{N × CPI_{avg}}{f} = $100000 \times 3.75/2.5 \times 10^9 = 375000/2.5 \times 10^9$

T = 0.00015 s

换算为微秒（1 s = 10^6 μs）：

T = 0.00015×10^6 = 150.00 μs

> **第2问答案：执行时间 T = 150.00 μs**

RESULT（最终结果）

问题	答案
第1问 平均 CPI	**3.75**
第2问 执行时间	**150.00 μs**

场景规则验证

规则	内容	验证结果
占比归一	Σ占比 = 35+30+25+10 = 100%	√
加权平均定义	CPI_avg = Σ(CPI_i × 占比_i)	√（见逐项计算）
平均 CPI 非整数	CPI_avg = 3.75，小数部分 = 0.75 ≠ 0	√
执行时间公式	$T = (N \times CPI_avg)/f = 375000/(2.5\times10^9)$	√
单位换算真实存在	GHz→Hz（$\times10^9$）、s→μs（$\times10^6$），两步非平凡换算	√
结果量级合理	T = 150 μs，落在 μs~ms 范围内	√

2.

数据寻址方式——有效地址（EA）计算 · 解答

已知参数

参数	值（十六进制）
形式地址 A	0x3C
基址寄存器 R_0	0x28
变址寄存器 R_1	0x50
通用寄存器 R_2	0x5A
程序计数器 PC	0x12（取本条指令后已更新的值）
Mem[0x3C]	0x7E

地址空间：0x0000～0xFFFF（16位），按字编址，机器字长16位。

第1问（2分）

（1）直接寻址

公式：EA = A

计算：EA = A = **0x3C**

（2）间接寻址

公式：EA = 内存地址 A 处所存储的内容，即 EA = (A) = Mem[A]

间接寻址链仅一跳：Mem[A] 的值本身即为有效地址，不再进行二次间接。

计算：EA = Mem[A] = Mem[0x3C] = **0x7E**

第2问（2分）

（1）寄存器间接寻址（使用 R_2）

公式：EA = (R_2)，即指定寄存器 R_2 的内容

计算：EA = R_2 = **0x5A**

（2）基址寻址（使用 R_0，形式地址 A）

公式：EA = (R_0) + A

计算：EA = R_0 + A = 0x28 + 0x3C = **0x64**

第3问（2分）

（1）变址寻址（使用 R_1，形式地址 A）

公式：EA = (R_1) + A

计算：EA = R_1 + A = 0x50 + 0x3C = **0x8C**

（2）相对寻址（使用 PC，形式地址 A）

公式：EA = (PC) + A，其中 PC 为取本条指令后已更新的值

计算：EA = PC + A = 0x12 + 0x3C = **0x4E**

结果汇总

寻址方式	计算公式	EA（十六进制）
直接寻址	EA = A	**0x3C**
间接寻址	EA = Mem[A] = Mem[0x3C]	**0x7E**
寄存器间接寻址	EA = R_2	**0x5A**
基址寻址	EA = R_0 + A = 0x28 + 0x3C	**0x64**
变址寻址	EA = R_1 + A = 0x50 + 0x3C	**0x8C**
相对寻址	EA = PC + A = 0x12 + 0x3C	**0x4E**

场景规则验证

6种寻址方式的 EA 互不相同（0x3C、0x7E、0x5A、0x64、0x8C、0x4E），每种寻址方式可独立区分；所有 EA 均在地址空间范围 [0x0000, 0xFFFF] 内，地址合法。

3.

进位生成函数：

$G_i = A_i \cdot B_i$

物理含义： $G_i = 1$ 表示第 i 位的两个操作数位 A_i 和 B_i 同时为 1，此时该位本身就会**生成**一个向高位的进位，与低位传来的进位无关。

进位传递函数：

$P_i = A_i \oplus B_i$

物理含义： $P_i = 1$ 表示第 i 位的 A_i 和 B_i 恰好一个为 1、一个为 0（半加和为 1），此时若低位有进位传来，该位会把进位**传递**到更高位（即 $C_i = C_{i-1}$）。

递推基式：$C_i = G_i + P_i \cdot C_{i-1}$

C_1 的展开：

$C_1 = G_1 + P_1 \cdot C_0$

（C_1 本身仅含 C_0，无需进一步代入。）

C_2 的展开：

$C_2 = G_2 + P_2 \cdot C_1$

将 $C_1 = G_1 + P_1 \cdot C_0$ 代入：

$C_2 = G_2 + P_2 \cdot (G_1 + P_1 \cdot C_0)$

分配律展开：

$\boxed{C_2 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot C_0}$

> 验证（gen.py 机检）：展开结果不含中间进位 C_1，且 == 递推基式反复代入的结果。✔

逐级代入 $C_3 \to C_2 \to C_1$，每一步代入基式 $C_i = G_i + P_i \cdot C_{i-1}$：

· $C_3 = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot C_0$

将 C_3 代入 $C_4 = G_4 + P_4 \cdot C_3$，分配律展开：

$\boxed{C_4 = G_4 + P_4 \cdot G_3 + P_4 \cdot P_3 \cdot G_2 + P_4 \cdot P_3 \cdot P_2 \cdot G_1 + P_4 \cdot P_3 \cdot P_2 \cdot P_1 \cdot C_0}$

> 验证（gen.py 机检）： - 展开结果不含任何中间进位 $C_1/C_2/C_3$ ✔ - == 递推基式反复代入的结果 ✔ - 含最高阶传递项 $P_4 \cdot P_3 \cdot P_2 \cdot P_1 \cdot C_0$ ✔

原理阐释：为什么先行进位能消除行波延迟？

在行波进位（ripple-carry）加法器中，$C_i = G_i + P_i \cdot C_{i-1}$，每一位的进位**必须等待前一级进位 C_{i-1} 计算完成后才能开始计算**，形成 $C_0 \to C_1 \to C_2 \to C_3 \to C_4$ 的串行链路，延迟随位数线性增长。

而在先行进位（CLA）中，经过上述展开后可以看到：

$C_4 = G_4 + P_4\,G_3 + P_4\,P_3\,G_2 + P_4\,P_3\,P_2\,G_1 + P_4\,P_3\,P_2\,P_1\,C_0$

每一个进位 C_i（包括 C_4）都仅依赖于 C_0 和本组的 $G_1 \sim G_4$、$P_1 \sim P_4$，表达式中不再出现任何中间进位变量 $C_1/C_2/C_3$。这意味着：

· 只要知道了所有位的 G_i 和 P_i（这些可由 A_i, B_i 在一级门延迟内并行算出）以及初始进位 C_0，就能**同时（并行）**计算出 C_1, C_2, C_3, C_4；

· **无需等待前一级进位逐级到达**，从而消除了行波进位的逐级延迟，将进位延迟从 O(n) 降低到常数级（由展开式的最大乘积项深度决定）。

这就是先行进位（Carry-Lookahead）的核心思想：用空间换时间，通过增加逻辑门的扇入和数量，换取进位的并行计算。

4.

参数（来自题干）

参数	值
总线数据宽度 W	64 bit
总线时钟频率 f	250 MHz
每完成一次数据传输所需的时钟周期数 n	3

带宽公式

$B = W/8 \times \frac{f \times 10^6}{n} \quad (MB/s)$

其中 W 单位为 bit（÷8 转 Byte），f 单位为 MHz（×10^6 转 Hz），n 为每传输周期占用的时钟周期数。

逐步代入

Step 1　bit → Byte 换算：

$W/8 = \frac{64\ bit}{8} = 8\ Byte$

Step 2　MHz → Hz 换算：

$f = 250\ MHz = 250 \times 10^6\ Hz = 250\ 000\ 000\ Hz$

Step 3　÷n（每秒传输次数）：

每完成一次数据传输需 n=3 个时钟周期，故每秒可完成的传输次数为：

$f/n = 250\ 000\ 000/3\ 次/s$

Step 4　计算带宽：

$B = 8\ Byte \times 250\ 000\ 000/3\ 次/s = 2\ 000\ 000\ 000/3\ Byte/s$

转为 MB/s（$1\ MB = 10^6\ Byte$）：

$B = 2\ 000\ 000\ 000/3 \times 1\ 000\ 000\ MB/s = 2000/3\ MB/s \approx 666.67\ MB/s$

最终结果

$\boxed{B = \dfrac{2000}{3}\ MB/s \approx 666.67\ MB/s}$

5.

基本信息

- 字长 n = 8，补码定点整数，表示范围 [−128, +127]
- [A]补 = 0x9C，A = −100
- [B]补 = 0xD8，B = −40

第(1)题：计算 [A + B]补 并判断溢出

运算过程

真实值验证：A + B = (−100) + (−40) = **−140**，超出表示范围 [−128, +127]，预期溢出。

模 2^8 加法：

```
  0x9C
+ 0xD8
-------
  0x174    ← 最高位向第 9 位进位
```

截断到 8 位（mod 256）：0x174 & 0xFF = **0x74**

溢出判断（两种方法互相印证）

① 双符号位法（变形补码法）

将 8 位补码符号扩展为双符号位：

操作数	变形补码（9~10 位）
[A]补 = 0x9C	1_1001_1100（符号扩展，补码 0x19C = 412）
[B]补 = 0xD8	1_1101_1000（符号扩展，补码 0x1D8 = 472）

```
  1_1001_1100   (0x19C = 412)
+ 1_1101_1000   (0x1D8 = 472)
──────────────
 11_0111_0100   (加法原始值 = 0x374 = 884)
```

截断到 9 位双符号位域（mod 2^9 = 取低 9 位）：0x374 & 0x1FF = **0x174**，即 `1_0111_0100`。

双符号位 = **10**（s_hi = 1, s_lo = 0，两位不一致）。

> 双符号位 `10`：符号位高位为 1、低位为 0，两者不一致 → **溢出**。（`10` 表示两个负数相加结果本应为负，但符号位出现 `10` → 负溢出/下溢）

② 进位判别法

- 最高位进位 Cs = 1（0x9C + 0xD8 向第 9 位进位）
- 次高位进位 Cp = 0（低 7 位 0x1C + 0x58 = 0x74，未向 bit7 进位）
- Cs ⊕ Cp = 1 ⊕ 0 = **1** → **溢出**

两种方法结论一致。

结果

[A + B]_补 = \boxed{0x74}

溢出：是（两个负数相加，真实值 −140 超出下界 −128，截断后变为正数 +116，符号翻转）

第(2)题：计算 [A − B]补 并判断溢出

第一步：求 [−B]补

[−B]_补 = \sim[B]_补 + 1 = \sim0xD8 + 1 = 0x27 + 1 = \boxed{0x28}

验证：[−B]补 + [B]补 = 0x28 + 0xD8 = 0x100，mod 2^8 = 0 √（加法逆元关系成立）

第二步：[A]补 + [−B]补

真实值验证：A − B = (−100) − (−40) = **−60**，在表示范围 [−128, +127] 内，预期不溢出。

模 2^8 加法：

```
  0x9C   ([A]补)
+ 0x28   ([-B]补)
-------
  0x0C4  ← 无向第 9 位的进位（最高位进位 = 0）
```

截断到 8 位：**0xC4**（十进制补码解释为 −60，与真实值一致 √）

溢出判断（两种方法互相印证）

① 双符号位法（变形补码法）

操作数	变形补码
[A]补 = 0x9C	1_1001_1100（0x19C = 412）
[−B]补 = 0x28	0_0010_1000（0x028 = 40）

```
  1_1001_1100
+ 0_0010_1000
──────────────
  1_1100_0100
```

双符号位 = **11** → **11**（符号位为 `11`）。

> 双符号位 `11`：两位一致 → **不溢出**（结果仍为负，与负数+正数的结果符号一致）

② 进位判别法

- 最高位进位 Cs = 0（0x9C + 0x28 = 0xC4，无向第 9 位进位）
- 次高位进位 Cp = 0（低 7 位 0x1C + 0x28 = 0x44，未向 bit7 进位）
- Cs ⊕ Cp = 0 ⊕ 0 = **0** → **不溢出**

两种方法结论一致。

结果

$[-B]_{补} = \boxed{0x28}, \qquad [A - B]_{补} = \boxed{0xC4}$

溢出：否（真实值 −60 在 [−128, +127] 内，结果正确为负数）

总结

	真实值	在范围内?	补码结果	溢出
第(1)题 A+B	−140	否（超下界）	0x74 (+116)	**是**
第(2)题 A−B	−60	是	0xC4 (−60)	**否**

两题形成"一溢一不溢"的对照： - 第(1)题两个负数相加，真实结果 −140 超出下界，截断后符号由负变正 → 溢出。 - 第(2)题负数减负数（等价于加正数），真实结果 −60 仍在范围内 → 不溢出。

两种溢出判断方法（双符号位法、进位判别法）对两题均给出一致结论。

四、应用题（10分）

1.

问题	答案
Q1 直接映射	Tag=12位（位23~12），行号=7位（位11~5），偏移=5位（位4~0）
Q2 4路组相联	Tag=14位（位23~10），组号=5位（位9~5），偏移=5位（位4~0）
Q3 直接映射	映射到第 34 行，Tag=291
Q3 组相联	映射到第 2 组，Tag=1165

> 场景规则验证（exec 全部通过）：SR-1 偏移=5位 √；SR-2 直接映射 12+7+5=24 √；SR-3 组相联 14+5+5=24 √；SR-4 Tag 差 2 位 ⩾1 √；SR-5 地址 0x123450 < 2^{24} √。

五、设计题（2×10=20分）

1.

- **第 1 问**：操作控制字段各字段位数为 第1组=3、第2组=2、第3组=3、第4组=2、第5组=3，**总位数 = 13 位**。
- **第 2 问**：微指令字长 = 13 + 2 + 9 = **24 位**。
- **第 3 问**：控制存储器最大容量 = **12288 位 = 1536 字节（1.5 KB）**。

2.

子问	核心答案
(1) 扩展方式	字位同时扩展；每组 2 片，共 8 组，总 16 片
(2) 地址线分配	片内 A_0~A_{11}（12根）；片选 A_{12}~A_{14}（3根）；剩余 A_{15}（1根）
(2) 地址范围	第 0~7 组：0x0000~0x0FFF, 0x1000~0x1FFF, ..., 0x7000~0x7FFF
(3) 片选逻辑	$\overline{CS_i} = \overline{Y_i}$，$A_{14}A_{13}A_{12}$ = i 的二进制（000~111）
(3) A_{15} 处理	作为译码器使能信号，避免地址镜像重复